DD: SUB-WF-UPGRADE-01 — Subscription Upgrade

Developer implementation guide: `DD_SUB-WF-UPGRADE-01-Subscription_Upgrade-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the module contract `DD_SUB-WF-UPGRADE-01-Subscription_Upgrade-v1.0.md`.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0 Operation that transitions an `ACTIVE` subscription's `package_ref` from a lower-priced Package to a higher-priced Package on the same HomePass. Owns: the 2-trigger taxonomy (`CUSTOMER_REQUESTED_UPGRADE` / `ADMIN_UPGRADE`), the target-package validation rules (same currency, price ≥ current, ACTIVE status, HomePass service-class compatibility), the equipment-compatibility check (new equipment binding only if the new Package requires Services not provided by the existing binding), the per-operator config catalog, the trigger REST endpoint, the rich domain Kafka events. Composes the framework primitives from **DD_SUB-WF-FRAMEWORK-01** + **DD_SUB-WF-FRAMEWORK-01-WORKERS**. **Effective timing — customer choice per request.** Three modes: `IMMEDIATE` (upgrade takes effect at saga commit), `END_OF_CURRENT_CYCLE` (upgrade fires at the cycle close timestamp), `SCHEDULED_AT` (upgrade fires at an arbitrary future timestamp within `subscription_upgrade_config.max_future_scheduled_days`). The BPMN parks on a Camunda intermediate timer until the chosen moment, then runs the same validate-bill-fulfill commit sequence. The customer chooses **independently** whether to PRESERVE or RESET the cycle anchor — though some combinations are redundant (see R-EF-3). Scheduled-future upgrades are cancellable via DELETE on the operation id; cancellation stops the timer and closes the operation as `CANCELLED`. Cycle anchor + cycle model effects per `cycleAnchorPolicy` apply at the actual commit time, not at the request time. **Companion DDs** (per-operator BPMN satellites): - **DD_SUB-WF-UPGRADE-01-FLOW-WIK** — KE WIK reference BPMN (shipped v1.0) - **DD_SUB-WF-UPGRADE-01-FLOW-WUG / -WTZ / -YASSN** — pending operator deep dives

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/04_subscription_workflows/DD_SUB-WF-UPGRADE-01-Subscription_Upgrade-v1.0.md`
Version	`v1.0`
DD type	`module contract`
BPMN/process keys	Not explicitly defined
Drools packages	`rules.subscription.upgrade.<operator>`

4. Runtime Contracts To Implement

4.1 APIs

- `DELETE /api/subscriptions/{subscription_id}/pending-upgrade/{operation_id}`
- `DELETE /pending-upgrade/{operationId}`
- `POST /api/subscriptions/{subscription_id}/upgrade`

4.2 Tables

- `subscription_upgrade_config`

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `bil01-emit-intent`
- `drools-decide`
- `external-cancel-requested`
- `ful-call-modify`
- `kafka-emit`
- `notification-send`
- `osr-equipment-bind`
- `osr-equipment-unbind`
- `payment-confirmed`
- `sub-lm-commit-state`
- `sub-lm-compute-cycle-window`
- `sub-lm-put-master-fields`
- `sub-lm-put-pending-status`
- `subscription-operation-finalize`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- `SubscriptionUpgradeCancelled`
- `SubscriptionUpgradeRejected`
- `SubscriptionUpgradeScheduled`
- `SubscriptionUpgraded`

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.

- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0

Operation that transitions an `ACTIVE` subscription's `package_ref` from a lower-priced Package to a higher-priced Package on the same HomePass. Owns: the 2-trigger taxonomy (`CUSTOMER_REQUESTED_UPGRADE` / `ADMIN_UPGRADE`), the target-package validation rules (same currency, price ≥ current, ACTIVE status, HomePass service-class compatibility), the equipment-compatibility check (new equipment binding only if the new Package requires Services not provided by the existing binding), the per-operator config catalog, the trigger REST endpoint, the rich domain Kafka events. Composes the framework primitives from **DD_SUB-WF-FRAMEWORK-01** + **DD_SUB-WF-FRAMEWORK-01-WORKERS**.

Effective timing — customer choice per request. Three modes: `IMMEDIATE` (upgrade takes effect at saga commit), `END_OF_CURRENT_CYCLE` (upgrade fires at the cycle close timestamp), `SCHEDULED_AT` (upgrade fires at an arbitrary future timestamp within `subscription_upgrade_config.max_future_scheduled_days`). The BPMN parks on a Camunda intermediate timer until the chosen moment, then runs the same validate-bill-fulfill commit sequence. The customer chooses **independently** whether to PRESERVE or RESET the cycle anchor — though some combinations are redundant (see R-EF-3). Scheduled-future upgrades are cancellable via DELETE on the operation id; cancellation stops the timer and closes the operation as `CANCELLED`. Cycle anchor + cycle model effects per `cycleAnchorPolicy` apply at the actual commit time, not at the request time.

Companion DDs (per-operator BPMN satellites):

- **DD_SUB-WF-UPGRADE-01-FLOW-WIK** — KE WIK reference BPMN (shipped v1.0)

- **DD_SUB-WF-UPGRADE-01-FLOW-WUG / -WTZ / -YASSN** — pending operator deep dives

Glossary

Term	Meaning
Upgrade	Change of the subscription's Package reference to a higher-priced Package on the same HomePass. Distinct from MIGRATION (different HomePass) and DOWNGRADE (lower price).
Source Package	The current `package_ref` + `package_version_id` snapshot at upgrade time. Pinned to the row for audit; also written to `previous_package_ref` on the subscription.
Target Package	The Package the subscription is upgrading to. Must be `ACTIVE` in SIP-01, same currency as the subscription, price ≥ source Package price, and every service family it requires must be present in the HomePass's `service_management_endpoints` (RLM-CFG-01) with a non-null port.
Proration delta	The amount BIL-01 computes for the remaining days of the current cycle at the price difference between target and source Packages. Owned by BIL-01 + BIL-CFG-01.
Equipment requirement delta	The set of Services in the target Package's composition that are NOT in the source Package's composition. If any of these new Services require equipment binding (per PLM-CFG-01) AND the current binding doesn't cover them, the workflow runs the equipment-bind step.
`cycleAnchorPolicy`	Customer/admin choice captured per upgrade request. Two values: `PRESERVE` (keep existing cycle anchor; charge prorated delta for cycle-days-remaining at price difference) or `RESET_TO_UPGRADE_DATE` (cycle anchor moves to upgrade commit date; charge a fresh first-cycle at target Package price; old cycle's accrued days converted to credit per BIL-01 policy). Operator config may default the value; the request may override. Applied at actual commit time (not request time).
`effectiveTiming`	Customer choice per request: `IMMEDIATE` (commit at saga start), `END_OF_CURRENT_CYCLE` (commit at the current cycle's `cycle_end`), `SCHEDULED_AT` (commit at a customer-supplied future timestamp). For non-IMMEDIATE modes, the BPMN parks on a Camunda intermediate timer until the chosen moment, then proceeds with validate-bill-fulfill-commit. The subscription remains `ACTIVE` on the source Package during the wait; charges continue at source price.
Pending-future upgrade	A scheduled upgrade in its wait state. Represented by the BPMN process instance parked on the timer + the `subscription_operation` row with `current_state = SCHEDULED`. Cancellable via DELETE on the operation id.

1. What this is

The upgrade operation. SUB-WF-UPGRADE-01 is one of the 10 operation kinds in the framework. Triggered by: a customer requesting an upgrade (CC, portal, retail) or an admin upgrading on the customer's behalf (e.g., as part of a retention play).

Outcomes (at actual commit time, which may be NOW or a future moment per `effectiveTiming`): subscription `package_ref` + `package_version_id` updated to target Package, `previous_package_ref` snapshot retained, prorated delta OR fresh first-cycle charge applied per `cycleAnchorPolicy` (pay-first invariant at commit time), network reconfigured for new Service profile (FUL-03 `MODIFY_SERVICE`), equipment-bind step run if equipment requirements changed, cycle anchor preserved or reset per the operation's chosen policy, customer notified.

For non-IMMEDIATE `effectiveTiming`, the subscription remains `ACTIVE` on the source Package during the wait. A second pending-future upgrade against the same subscription is rejected; any state-affecting operation (PAUSE, TERMINATE, another UPGRADE) that fires during the wait cancels the pending upgrade per R-EF-5.

What this DD owns

1. `subscription_upgrade_config` **table** — per-operator config: customer-self-service enablement, pay-first-gate behavior.
2. **Trigger taxonomy** — 2 triggers (`CUSTOMER_REQUESTED_UPGRADE` / `ADMIN_UPGRADE`) with role-gating.
3. **Target-package validation rules** — currency match, status ACTIVE, price ≥ source, HomePass compatibility, equipment-requirement delta computation.
4. `cycleAnchorPolicy` **taxonomy** — 2 values (`PRESERVE` / `RESET_TO_UPGRADE_DATE`) carried per request; defaulted by operator config when not supplied.

5. `effectiveTiming` **taxonomy** — 3 values (`IMMEDIATE` / `END_OF_CURRENT_CYCLE` / `SCHEDULED_AT`); for non-IMMEDIATE, the BPMN parks on a Camunda intermediate timer; cancellable.
6. **Trigger REST endpoints** — `POST /api/subscriptions/{subscription_id}/upgrade` to start an upgrade; `DELETE /api/subscriptions/{subscription_id}/pending-upgrade/{operation_id}` to cancel a pending-future upgrade.
7. **Domain Kafka events** — `SubscriptionUpgradeScheduled` (for non-IMMEDIATE acceptance), `SubscriptionUpgraded` (at commit), `SubscriptionUpgradeRejected` (failure), `SubscriptionUpgradeCancelled` (cancellation of pending-future).
8. **Upgrade workflow rules** — 37 rules in §4.

What this DD does NOT own

- **BPMN files** — owned by per-operator satellites.
- **Worker implementations** — catalogued in DD_SUB-WF-FRAMEWORK-01-WORKERS.
- **Subscription state machine** — owned by SUB-LM-01.
- **Package catalog** — owned by SIP-01.
- **Service catalog + service-class definitions + equipment-requirement declarations** — owned by PLM-CFG-01.
- **HomePass row mutations** — owned by RLM-CFG-01. This workflow reads `service_management_endpoints`, `status_code`, and the status-code catalog; it does not mutate any HomePass column. If the target Package requires a service family the HomePass doesn't physically support, the upgrade is blocked at validation — caller must use SUB-WF-MIGRATION-01.
- **Upgrade-prorate delta math + BillableEvent definitions** — owned by BIL-CFG-01 + BIL-01.
- **Network reconfiguration + per-service-class provisioner fan-out** — owned by FUL-03 under `MODIFY_SERVICE`. This workflow sends FUL-03 the source + target Package snapshots; FUL-03 computes the service-class diff and dispatches the appropriate provisioner per service class (modifyService for unchanged, activate for new, deactivate for obsolete when unbindObsoleteEquipment). Network-side per-customer activation (IGMP-snooping, multicast group provisioning, AAA channel-package registration) is FUL-03's provisioner-adapter responsibility.
- **Equipment binding mechanics** — owned by OSR-INSTANCE-01.
- **Restrictions during upgrade** — preserved as-is on the subscription's `active_restrictions[]` array; this DD does NOT clear or re-apply restrictions.
- **Customer notification template + channel** — owned by NOT-01.

Regional flexibility

Per-operator levers:

Lever	Examples for UPGRADE
BPMN file	KE: standard BPMN with conditional equipment-bind step + pay-first gate. Other operators may add a supervisor-approval user task for upgrades above a threshold.
Drools package `rules.subscription.upgrade.<operator>`	KE: target-package validation, equipment-requirement-delta computation, self-service eligibility.
`subscription_upgrade_config` row	KE: customer self-service enabled.
FOUNDATION_AUTH candidate groups	Operator-specific resolution.

2. Tables overview

This DD owns one table: `subscription_upgrade_config` (per-operator config).

Cross-operation audit anchor: `subscription_operation` (framework-owned).

Subscription master fields updated (owned by SUB-LM-01): `package_ref`, `package_version_id`, `previous_package_ref`, `previous_package_version_id`, `last_status_changed_at`. `current_transition_type = UPGRADE` during PENDING_UPGRADE.

3. Schemas

3.1 `subscription_upgrade_config`

One row per operator.

```
CREATE TABLE subscription_upgrade_config (
  operator_code                          TEXT PRIMARY KEY,
  customer_self_service_enabled          BOOLEAN NOT NULL DEFAULT FALSE,
  pay_first_required                     BOOLEAN NOT NULL DEFAULT TRUE,
  default_cycle_anchor_policy            TEXT NOT NULL DEFAULT 'PRESERVE',
  default_effective_timing               TEXT NOT NULL DEFAULT 'IMMEDIATE',
  max_future_scheduled_days              INTEGER NOT NULL DEFAULT 90,
  customer_notification_enabled          BOOLEAN NOT NULL DEFAULT TRUE,
  updated_at                             TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by                             TEXT NOT NULL,
  CHECK (default_cycle_anchor_policy IN ('PRESERVE','RESET_TO_UPGRADE_DATE')),
  CHECK (default_effective_timing IN ('IMMEDIATE','END_OF_CURRENT_CYCLE','SCHEDULED_AT')),
  CHECK (max_future_scheduled_days > 0)
);
```

Column	Notes
`customer_self_service_enabled`	When `true`, the `CUSTOMER_REQUESTED_UPGRADE` trigger may be initiated from the customer portal (`CUSTOMER_SELF_SERVICE` role). When `false`, only `CALL_CENTER` and admin roles may initiate it.
`pay_first_required`	When `true` (default), the proration delta from BIL-01's intent decision is collected via the pay-first gate (BPMN waits for `payment-confirmed` before proceeding). Applies at actual commit time, not request time. When `false`, the charge is invoiced post-commit. KE v1.0: `true`.
`default_cycle_anchor_policy`	Default `cycleAnchorPolicy` applied when the request does not supply one. `PRESERVE` keeps the existing cycle anchor (BIL-01 charges prorated delta for cycle-days-remaining). `RESET_TO_UPGRADE_DATE` moves cycle anchor to commit date (BIL-01 issues a fresh first-cycle charge at target price; remaining old-cycle days converted to credit per BIL-01 policy).
`default_effective_timing`	Default `effectiveTiming` applied when the request does not supply one. `IMMEDIATE` commits at saga start. `END_OF_CURRENT_CYCLE` parks until `cycle_end` then commits. `SCHEDULED_AT` requires the request to also supply `scheduledAt`.
`max_future_scheduled_days`	Maximum window (days from request time) for `SCHEDULED_AT.scheduledAt`. Requests beyond this return 400 `SCHEDULED_AT_BEYOND_MAX_WINDOW`. Caps abuse + parked-timer count.
`customer_notification_enabled`	When `true`, the workflow calls `notification-send` at acceptance (for scheduled) AND at commit. NOT-01 owns template, channel, language, fallback.

Sample data:

operator_code	customer_self_service_enabled	pay_first_required	default_cycle_anchor_policy	default_effective_timing	max_future_scheduled_days	customer_notification_enabled
`WIK`	true	true	`PRESERVE`	`IMMEDIATE`	90	true
`WUG`	false	true	`PRESERVE`	`IMMEDIATE`	90	true
`WTZ`	false	true	`PRESERVE`	`IMMEDIATE`	90	true
`YASSN`	false	true	`PRESERVE`	`IMMEDIATE`	90	true

4. Rules

4.1 State precondition + transition

Rule	Description
R-SUB-WF-UPGRADE-01-S-1	Upgrade requires the subscription's `status_code = ACTIVE` at request time AND at commit time. Other statuses return 409 `INVALID_STATE_FOR_UPGRADE`. For scheduled-future upgrades, the check at commit time may fail if the subscription was paused or terminated during the wait — handled by R-EF-5.
R-SUB-WF-UPGRADE-01-S-2	The transition at commit is `ACTIVE` → `PENDING_UPGRADE` → `ACTIVE`. `PENDING_UPGRADE` is the framework-internal transient state during the commit sequence. For scheduled-future upgrades, the subscription stays `ACTIVE` during the wait — PENDING_UPGRADE is entered only when the timer fires.

Rule	Description
R-SUB-WF-UPGRADE-01-S-3	Per framework R-FW-1, upgrade COMMIT is mutually exclusive with other state-affecting operations on the same subscription. A second upgrade COMMIT cannot fire while another commit is in progress. A scheduled-future upgrade in its wait state DOES allow other operations to fire (e.g., a customer-requested PAUSE) — see R-EF-5 for cancellation semantics.
R-SUB-WF-UPGRADE-01-S-4	The subscription's `current_transition_type` is set to `UPGRADE` during PENDING_UPGRADE. `current_transition_reason_code` is set to the supplied `upgradeTrigger`. NOT set during scheduled-future wait.
R-SUB-WF-UPGRADE-01-S-5	Restrictions on the subscription (`active_restrictions[]`) are NEITHER cleared NOR validated by this workflow. A customer with a restriction (e.g., `OUTGOING_VOICE_BARRED` from BIL-04 L2 dunning) MAY upgrade — the restriction persists post-upgrade.
R-SUB-WF-UPGRADE-01-S-6	At most one pending-future upgrade per subscription. A second non-IMMEDIATE upgrade request returns 409 `PENDING_UPGRADE_ALREADY_SCHEDULED` with the existing operation id. An IMMEDIATE upgrade request while a pending-future upgrade exists returns 409 `PENDING_UPGRADE_ALREADY_SCHEDULED` (caller must cancel the pending one first or wait).

4.2 Trigger taxonomy + role gating

Rule	Description
R-SUB-WF-UPGRADE-01-RT-1	2 triggers: `CUSTOMER_REQUESTED_UPGRADE` / `ADMIN_UPGRADE`. Carried in the request body as `upgradeTrigger`. Unknown values return 400 `UNKNOWN_UPGRADE_TRIGGER`.
R-SUB-WF-UPGRADE-01-RT-2	Role-trigger gating: `CUSTOMER_REQUESTED_UPGRADE` requires `CUSTOMER_SELF_SERVICE` OR `CALL_CENTER` OR `SUBSCRIPTION_OPERATOR`; `ADMIN_UPGRADE` requires `SUBSCRIPTION_ADMIN` OR `SUBSCRIPTION_OPERATOR`. Mismatched role returns 403 `ROLE_NOT_AUTHORIZED_FOR_TRIGGER`.
R-SUB-WF-UPGRADE-01-RT-3	`CUSTOMER_SELF_SERVICE` role allowed only when `subscription_upgrade_config.customer_self_service_enabled = true`. Else 403.

4.3 Target-package validation

Rule	Description
R-SUB-WF-UPGRADE-01-TP-1	`targetPackageRef` must reference a `package` row with `status = ACTIVE` in SIP-01. `DRAFT`, `INACTIVE`, or `END_OF_LIFE` Packages return 400 `TARGET_PACKAGE_NOT_ACTIVE`.
R-SUB-WF-UPGRADE-01-TP-2	The target Package's current active `package_version` row supplies the price + currency snapshot. Currency MUST match the subscription's currency. Cross-currency upgrades return 400 `CURRENCY_MISMATCH`.
R-SUB-WF-UPGRADE-01-TP-3	Target Package's current active version price MUST be ≥ source Package's pinned version price (R-WF-3 of v0.9 wording). Lower-priced target returns 400 `NOT_AN_UPGRADE` with guidance to use SUB-WF-DOWNGRADE-01.
R-SUB-WF-UPGRADE-01-TP-4	Target Package MUST be different from source Package (`targetPackageRef ≠ subscription.package_ref`). Same-package return 400 `SAME_PACKAGE`.
R-SUB-WF-UPGRADE-01-TP-5	The current HomePass MUST be physically capable of carrying every service family required by the target Package. Specifically: for each Service in the target Package's `package_composition`, the Service's `serviceManagementKey` (e.g., `data`, `voice`, `iptv_multicast`) MUST be present in `homepass.service_management_endpoints` keys. RLM-CFG-01 derives this map from `networkPath` at save time. If a required key is absent, return 400 `PACKAGE_NOT_AVAILABLE_AT_HOMEPASS` with the missing service-management keys and guidance to use SUB-WF-MIGRATION-01.
R-SUB-WF-UPGRADE-01-TP-6	For each required `serviceManagementKey`, the resolved endpoint's `port` MUST NOT be null (per RLM-CFG-01 — null port means `AT_INSTALLATION` mode, install hasn't run). Null port returns 400 `HOMEPASS_PORT_NOT_PROVISIONED` with guidance to complete install first.
R-SUB-WF-UPGRADE-01-TP-7	The HomePass `status_code` MUST be in a status with `is_active = true` per the operator's `homepass_status_code` catalog (RLM-CFG-01). HomePass in retired, suspended, or not-yet-serviceable status returns 409 `HOMEPASS_NOT_ACTIVE` with the current HomePass status.

Rule	Description
R-SUB-WF-UPGRADE-01-TP-8	Target Package's billing-frequency-days SHOULD match the source Package's. Mismatch is not rejected but produces a warning event field `billingFrequencyChanged=true`; BIL-01 handles the cycle implications per its policy.

4.4 Equipment-requirement delta

Rule	Description
R-SUB-WF-UPGRADE-01-EQ-1	The workflow computes the set of `service_class` values in the target Package that are NOT in the source Package. For each new service class, if PLM-CFG-01 declares it `requires_equipment = true` AND no existing equipment binding (OSR-INSTANCE-01) covers it, the BPMN runs an equipment-bind step. Otherwise, the step is skipped.
R-SUB-WF-UPGRADE-01-EQ-2	If equipment-bind is required, the trigger endpoint MUST include `equipmentBindings` (array of `{equipmentId, serviceClassId, fulfillsServiceClass}`). Missing bindings for required service classes return 400 `EQUIPMENT_BINDINGS_REQUIRED` with the list of unfulfilled service classes.
R-SUB-WF-UPGRADE-01-EQ-3	Equipment from the source Package's composition (existing bindings) is retained unless the workflow's `unbindObsoleteEquipment` flag is set (defaults to `false`). When set to `true` AND the target Package no longer requires that service class, the BPMN runs an unbind step (releases the equipment back to inventory). KE v1.0 defaults to `false` (retain existing equipment for customer convenience).

4.5 Billing + fulfillment

Rule	Description
R-SUB-WF-UPGRADE-01-B-1	Workflow calls `bil01-emit-intent` with `intentKind = UPGRADE_INTENT`. Context: `{sourcePackageRef, sourcePackageVersionId, sourcePackagePrice, targetPackageRef, targetPackageVersionId, targetPackagePrice, billingFrequencyChanged, upgradeTrigger, cycleAnchorPolicy}`. BIL-01 computes the appropriate charge per `cycleAnchorPolicy` (see R-CA-3), may apply BillableEvents per BIL-CFG-01.
R-SUB-WF-UPGRADE-01-B-2	If `pay_first_required = true` AND BIL-01's `totals.net > 0`: pay-first gate. The BPMN waits for `payment-confirmed` before proceeding. Payment timeout (7 days) → revert to source Package + ACTIVE.
R-SUB-WF-UPGRADE-01-B-3	FUL-03 called with `intent=MODIFY_SERVICE`, passing the full source + target Package snapshots. FUL-03 is responsible for computing the service-class fan-out from the diff: (a) Service classes in both source and target → provisioner `modifyService` (or no-op if Service profile unchanged at this granularity); (b) service classes in target but NOT source → provisioner `activate` for that service class, dispatched via the HomePass's `service_management_endpoints[serviceManagementKey]`; (c) service classes in source but NOT target → provisioner `deactivate` ONLY when `unbindObsoleteEquipment = true`, else retained (degenerate fulfillment with vestigial provisioning is acceptable per operator policy). FUL-03 also handles per-customer network-side activation tasks (IGMP-snooping config, multicast group provisioning, AAA channel-package registration) under the relevant provisioner's `activate` for each new service class.
R-SUB-WF-UPGRADE-01-B-4	**HomePass row is NOT mutated by this workflow.** The HomePass's `servicesSupported`, `service_management_endpoints`, and `status_code` are RLM-CFG-01-owned read-only inputs to this workflow. What changes during a service-class-expansion upgrade is *per-customer network-side provisioning* (FUL-03's responsibility), and *equipment binding* on the subscription (this DD's §4.4). The HomePass's physical capabilities (whether the OLT supports IPTV multicast, whether a VOIPSWITCH is on the path) pre-exist the upgrade — if they don't, the upgrade is blocked at R-TP-5/R-TP-6 and the caller must use MIGRATION-01 to move to a different HomePass.
R-SUB-WF-UPGRADE-01-B-5	On commit, master fields updated: `package_ref = targetPackageRef`, `package_version_id = targetPackageVersionId`, `previous_package_ref = sourcePackageRef`, `previous_package_version_id = sourcePackageVersionId`, `last_status_changed_at = NOW`. If `cycleAnchorPolicy = RESET_TO_UPGRADE_DATE`: `cycle_anchor_day` and `cycle_anchor_at` master fields are also updated to NOW (owned by SUB-LM-01; this DD's master-fields worker writes them).

4.6 Cycle anchor policy

Rule	Description
R-SUB-WF-UPGRADE-01-CA-1	The request supplies `cycleAnchorPolicy` ∈ {`PRESERVE`, `RESET_TO_UPGRADE_DATE`}. If absent, the workflow defaults to `subscription_upgrade_config.default_cycle_anchor_policy` for the operator. Unknown value returns 400 `UNKNOWN_CYCLE_ANCHOR_POLICY`.
R-SUB-WF-UPGRADE-01-CA-2	`cycleAnchorPolicy` is captured per upgrade — it is NOT a subscription-level invariant. A single subscription may have one upgrade with PRESERVE and a later upgrade with RESET_TO_UPGRADE_DATE; each is honored independently. The resolved policy is persisted to the Kafka event payload + the framework's `subscription_operation` audit row.
R-SUB-WF-UPGRADE-01-CA-3	BIL-01 interprets `cycleAnchorPolicy` in the `UPGRADE_INTENT` context. `PRESERVE` → BIL-01 returns a prorated delta line item (`UPGRADE_PRORATE` BillableEvent) for cycle-days-remaining × price-difference. `RESET_TO_UPGRADE_DATE` → BIL-01 returns a fresh first-cycle line item at target price (`UPGRADE_FIRST_CYCLE` BillableEvent) and may return a separate credit line for the unused portion of the old cycle (`UPGRADE_OLD_CYCLE_CREDIT` BillableEvent — BIL-01 policy decides whether to credit or absorb).
R-SUB-WF-UPGRADE-01-CA-4	When `RESET_TO_UPGRADE_DATE`, the new cycle starts at commit time and runs for `targetPackage.billing_frequency_days`. Subsequent cycles roll forward from the new anchor. Cycle-anchor mutation is committed via the master-fields step alongside `package_ref` (same atomic update).

4.7 Effective timing

Rule	Description
R-SUB-WF-UPGRADE-01-EF-1	The request supplies `effectiveTiming` ∈ {`IMMEDIATE`, `END_OF_CURRENT_CYCLE`, `SCHEDULED_AT`}. If absent, the workflow defaults to `subscription_upgrade_config.default_effective_timing`. Unknown value returns 400 `UNKNOWN_EFFECTIVE_TIMING`.
R-SUB-WF-UPGRADE-01-EF-2	When `effectiveTiming = SCHEDULED_AT`, the request MUST also supply `scheduledAt` (ISO 8601 timestamp). Missing → 400 `SCHEDULED_AT_MISSING`. `scheduledAt` MUST be strictly in the future AND within `subscription_upgrade_config.max_future_scheduled_days` from now. Past or beyond-window → 400 `SCHEDULED_AT_OUT_OF_BOUNDS`.
R-SUB-WF-UPGRADE-01-EF-3	Combination `effectiveTiming = END_OF_CURRENT_CYCLE` AND `cycleAnchorPolicy = RESET_TO_UPGRADE_DATE` is redundant (end-of-cycle commit IS the natural next anchor). Returns 400 `REDUNDANT_POLICY_COMBINATION` with guidance to use `cycleAnchorPolicy = PRESERVE` for end-of-cycle upgrades.
R-SUB-WF-UPGRADE-01-EF-4	For non-IMMEDIATE timings: the BPMN parks on a Camunda intermediate timer. The `subscription_operation.current_state` is set to `SCHEDULED`. The trigger endpoint returns 202 with `scheduledCommitAt` populated. `SubscriptionUpgradeScheduled` is emitted on Kafka. At the timer fire, the BPMN re-validates (status, target package still ACTIVE, HomePass still compatible, equipment still bound — same Drools decisions as IMMEDIATE) and proceeds with the commit sequence.
R-SUB-WF-UPGRADE-01-EF-5	Cancellation paths during the wait: (a) explicit `DELETE /pending-upgrade/{operationId}` from BO admin or customer — stops the timer, emits `SubscriptionUpgradeCancelled`, closes the operation as `CANCELLED`; (b) any state-affecting operation on the same subscription (PAUSE, SUSPEND-NP, TERMINATE, another UPGRADE) — the framework's `external-cancel-requested` mechanism stops the timer and emits `SubscriptionUpgradeCancelled` with `cancellationReasonCode = SUPERSEDED_BY_<otherOperationKind>`.
R-SUB-WF-UPGRADE-01-EF-6	At commit time (after the wait), the workflow re-validates all preconditions (S-1, TP-1 through TP-7, equipment delta still satisfied). If any check fails at commit, the workflow rejects with `SubscriptionUpgradeRejected` carrying `rejectionStage = COMMIT_REVALIDATION` and the failed rule code. Example: target Package transitioned to `END_OF_LIFE` between schedule and commit → reject.
R-SUB-WF-UPGRADE-01-EF-7	The `cycleAnchorPolicy` recorded at request time is honored at commit time. For `END_OF_CURRENT_CYCLE`, the commit naturally lands on the cycle boundary; BIL-01 returns zero proration delta (the current cycle finishes on source price, next cycle starts on target price); no anchor mutation needed (PRESERVE behavior). For `SCHEDULED_AT` mid-cycle commits, BIL-01 behaves identically to IMMEDIATE at the commit moment (proration delta if PRESERVE; old-cycle credit + fresh cycle if RESET_TO_UPGRADE_DATE — anchor moves to the scheduled date).

Rule	Description
R-SUB-WF-UPGRADE-01-EF-8	Pay-first gate behavior: for non-IMMEDIATE timings, the charge is pre-authorized at COMMIT time, not request time. The customer's wallet / card is only touched at commit. This matches the principle that scheduled upgrades shouldn't pre-collect money.

5. REST API

5.1 Trigger upgrade

```
POST /api/subscriptions/{subscription_id}/upgrade HTTP/1.1
Authorization: Bearer <jwt>
Content-Type: application/json
Idempotency-Key: upgrade-sub_01HZ_KAMAU-pkg_INET_250-20260520

{
  "operatorCode":          "WIK",
  "upgradeTrigger":        "CUSTOMER_REQUESTED_UPGRADE",
  "targetPackageRef":      "pkg_01HX_INET_250M_VOIP_RES",
  "cycleAnchorPolicy":     "PRESERVE",
  "effectiveTiming":       "IMMEDIATE",
  "scheduledAt":           null,
  "equipmentBindings":     [],
  "unbindObsoleteEquipment": false,
  "notes":                 "Customer requested speed upgrade after promotion notice",
  "initiatingActorUserId": "cc-agent-wik-007",
  "initiatingActorRole":   "CALL_CENTER",
  "initiatingWorkflowKind":"BO_UI",
  "initiatingWorkflowRef": "cc-ticket-2026-05-20-4421"
}

→ 202 Accepted
{
  "operationId":               "subop_01HZ_KAMAU_UPGRADE",
  "operatorCode":              "WIK",
  "subscriptionId":            "sub_01HZ_KAMAU_FIBER",
  "operationKind":             "UPGRADE",
  "upgradeTrigger":            "CUSTOMER_REQUESTED_UPGRADE",
  "resolvedCycleAnchorPolicy": "PRESERVE",
  "resolvedEffectiveTiming":   "IMMEDIATE",
  "scheduledCommitAt":         null,
  "bpmnProcessKey":            "sub-upgrade-wik",
  "bpmnProcessInstanceId":     "pi_01HZ_KAMAU_UPGRADE",
  "currentState":              "VALIDATING",
  "startedAt":                 "2026-05-20T14:00:00+03:00"
}
```

Target not active:

```
→ 400 Bad Request
{
  "code":              "TARGET_PACKAGE_NOT_ACTIVE",
  "targetPackageRef":  "pkg_01HX_INET_LEGACY",
  "targetPackageStatus": "END_OF_LIFE"
}
```

Not an upgrade (target cheaper):

```
→ 400 Bad Request
{
  "code":               "NOT_AN_UPGRADE",
  "sourcePackagePrice": 4500.00,
  "targetPackagePrice": 2500.00,
  "currency":           "KES",
  "guidance":           "Target Package is cheaper. Use SUB-WF-DOWNGRADE-01."
}
```

HomePass doesn't support target's services:

```
→ 400 Bad Request
{
  "code":               "PACKAGE_NOT_AVAILABLE_AT_HOMEPASS",
  "homePassId":         "hp_01HZ_KAREN_017",
  "missingServiceClasses": ["IPTV"],
  "guidance":           "Target Package requires IPTV which the current HomePass does not support. Use SUB-WF-MIGRATION-01
}
```

Equipment bindings required:

```
→ 400 Bad Request
{
  "code":                       "EQUIPMENT_BINDINGS_REQUIRED",
  "unfulfilledServiceClasses":  ["IPTV"],
  "currentBindings":            [{"serviceClassId":"INET","equipmentId":"eqp_ONT_42"}, {"serviceClassId":"VOIP","equipment
  "guidance":                   "Include equipmentBindings entries fulfilling the unfulfilled service classes."
}
```

Unknown cycle-anchor policy:

```
→ 400 Bad Request
{
  "code":                "UNKNOWN_CYCLE_ANCHOR_POLICY",
  "value":               "ROLL_FORWARD",
  "allowed":             ["PRESERVE","RESET_TO_UPGRADE_DATE"]
}
```

5.2 Scheduled-future upgrade — example

Customer wants to upgrade at end of current cycle, keep anchor:

```
POST /api/subscriptions/{subscription_id}/upgrade HTTP/1.1
Idempotency-Key: upgrade-sub_01HZ_KAMAU-pkg_INET_250-eoc-20260520

{
  "operatorCode":          "WIK",
  "upgradeTrigger":        "CUSTOMER_REQUESTED_UPGRADE",
  "targetPackageRef":      "pkg_01HX_INET_250M_VOIP_RES",
  "cycleAnchorPolicy":     "PRESERVE",
  "effectiveTiming":       "END_OF_CURRENT_CYCLE",
  "equipmentBindings":     [],
  "unbindObsoleteEquipment": false,
  "initiatingActorRole":   "CALL_CENTER",
  "initiatingActorUserId": "cc-agent-wik-007"
}

→ 202 Accepted
{
  "operationId":               "subop_01HZ_KAMAU_UPGRADE_EOC",
  "operationKind":             "UPGRADE",
  "resolvedCycleAnchorPolicy": "PRESERVE",
  "resolvedEffectiveTiming":   "END_OF_CURRENT_CYCLE",
  "scheduledCommitAt":         "2026-05-31T23:59:59+03:00",
  "currentState":              "SCHEDULED",
  "bpmnProcessInstanceId":     "pi_01HZ_KAMAU_UPGRADE_EOC"
}
```

Customer wants to upgrade on a specific future date, RESET anchor:

```
POST /api/subscriptions/{subscription_id}/upgrade HTTP/1.1
{
  "effectiveTiming":   "SCHEDULED_AT",
  "scheduledAt":       "2026-06-15T00:00:00+03:00",
  "cycleAnchorPolicy": "RESET_TO_UPGRADE_DATE",
  ...
}

→ 202 Accepted
{
  "scheduledCommitAt":        "2026-06-15T00:00:00+03:00",
  "resolvedEffectiveTiming":  "SCHEDULED_AT",
  "resolvedCycleAnchorPolicy":"RESET_TO_UPGRADE_DATE",
  "currentState":             "SCHEDULED"
}
```

Out-of-window scheduledAt:

```
→ 400 Bad Request
{
  "code":                  "SCHEDULED_AT_OUT_OF_BOUNDS",
  "scheduledAt":           "2026-12-15T00:00:00+03:00",
  "maxFutureScheduledDays": 90,
  "now":                   "2026-05-20T14:00:00+03:00"
}
```

Redundant policy combination:

```
→ 400 Bad Request
{
  "code":              "REDUNDANT_POLICY_COMBINATION",
  "effectiveTiming":   "END_OF_CURRENT_CYCLE",
  "cycleAnchorPolicy": "RESET_TO_UPGRADE_DATE",
  "guidance":          "END_OF_CURRENT_CYCLE commit naturally aligns the new anchor with cycle close. Use cycleAnchorPolic
```

```
}
```

Already-scheduled upgrade:

```
→ 409 Conflict
{
  "code":                   "PENDING_UPGRADE_ALREADY_SCHEDULED",
  "existingOperationId":    "subop_01HZ_KAMAU_UPGRADE_EOC",
  "existingScheduledCommitAt": "2026-05-31T23:59:59+03:00",
  "guidance":               "Cancel the existing scheduled upgrade before submitting a new one."
}
```

5.3 Cancel pending-future upgrade

```
DELETE /api/subscriptions/{subscription_id}/pending-upgrade/{operation_id} HTTP/1.1
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "operatorCode":          "WIK",
  "cancellationReasonCode":"CUSTOMER_CHANGED_MIND",
  "notes":                 "Customer called to cancel — wants to keep current plan",
  "initiatingActorUserId": "cc-agent-wik-007",
  "initiatingActorRole":   "CALL_CENTER"
}

→ 200 OK
{
  "operationId":           "subop_01HZ_KAMAU_UPGRADE_EOC",
  "cancellationReasonCode":"CUSTOMER_CHANGED_MIND",
  "cancelledAt":           "2026-05-22T11:30:00+03:00",
  "subscriptionStaysOn":   "pkg_01HX_INET_100M_VOIP_RES",
  "finalState":            "CANCELLED"
}
```

Not pending (already committed or doesn't exist):

```
→ 409 Conflict
{
  "code":            "OPERATION_NOT_PENDING",
  "currentState":    "COMPLETED",
  "guidance":        "Operation has already committed. To revert, submit a DOWNGRADE request."
}
```

6. Kafka events emitted

Event	Topic	Fires on	Consumers
`SubscriptionUpgradeScheduled`	`sophix.subscription.subscription.upgradescheduled`	Acceptance of non-IMMEDIATE timing (request approved, parked on timer)	NOT-01 (customer notification "your upgrade is scheduled for ..."), CVM (intent funnel), EM-04
`SubscriptionUpgraded`	`sophix.subscription.subscription.upgraded`	Successful commit (IMMEDIATE OR after timer fires)	NOT-01 (customer notification per operator config), BIL-02 (cycle accrual at new price), CVM (upgrade analytics), EM-04 reporting, SIP-01 (package-version-pinning audit)
`SubscriptionUpgradeCancelled`	`sophix.subscription.subscription.upgradecancelled`	Pending-future upgrade cancelled (explicit DELETE or superseded by another operation)	NOT-01, CVM, EM-04
`SubscriptionUpgradeRejected`	`sophix.subscription.subscription.upgraderejected`	Workflow failure (validation, billing, fulfillment, payment timeout, commit-time revalidation)	NOT-01, ICN-01 ops alert, EM-04

Sample `SubscriptionUpgradeScheduled`:

```
{
  "eventType":   "SubscriptionUpgradeScheduled",
  "aggregateId": "sub_01HZ_KAMAU_FIBER",
  "timestamp":   "2026-05-20T14:00:01+03:00",
  "payload": {
    "operatorCode":             "WIK",
    "subscriptionId":           "sub_01HZ_KAMAU_FIBER",
    "customerId":               "cust_01HZ_KAMAU",
    "operationId":              "subop_01HZ_KAMAU_UPGRADE_EOC",
    "upgradeTrigger":           "CUSTOMER_REQUESTED_UPGRADE",
    "sourcePackageRef":         "pkg_01HX_INET_100M_VOIP_RES",
    "targetPackageRef":         "pkg_01HX_INET_250M_VOIP_RES",
```

```
    "resolvedCycleAnchorPolicy": "PRESERVE",
    "resolvedEffectiveTiming":   "END_OF_CURRENT_CYCLE",
    "scheduledCommitAt":         "2026-05-31T23:59:59+03:00",
    "scheduledAt":               "2026-05-20T14:00:01+03:00",
    "initiatingActorUserId":     "cc-agent-wik-007",
    "initiatingActorRole":       "CALL_CENTER"
  }
}
```

Sample `SubscriptionUpgradeCancelled`:

```
{
  "eventType":   "SubscriptionUpgradeCancelled",
  "aggregateId": "sub_01HZ_KAMAU_FIBER",
  "payload": {
    "operationId":           "subop_01HZ_KAMAU_UPGRADE_EOC",
    "cancellationReasonCode":"CUSTOMER_CHANGED_MIND",
    "cancelledAt":           "2026-05-22T11:30:00+03:00",
    "subscriptionStaysOn":   "pkg_01HX_INET_100M_VOIP_RES",
    "originalScheduledCommitAt": "2026-05-31T23:59:59+03:00"
  }
}
```

Sample `SubscriptionUpgraded` (IMMEDIATE — same as before):

```
{
  "eventType":   "SubscriptionUpgraded",
  "aggregateId": "sub_01HZ_KAMAU_FIBER",
  "timestamp":   "2026-05-20T14:02:18+03:00",
  "payload": {
    "operatorCode":            "WIK",
    "subscriptionId":          "sub_01HZ_KAMAU_FIBER",
    "customerId":              "cust_01HZ_KAMAU",
    "operationId":             "subop_01HZ_KAMAU_UPGRADE",
    "upgradeTrigger":          "CUSTOMER_REQUESTED_UPGRADE",
    "sourcePackageRef":        "pkg_01HX_INET_100M_VOIP_RES",
    "sourcePackageVersionId":  "pkv_01HX_INET_100M_V3",
    "sourcePackagePrice":      2500.00,
    "targetPackageRef":        "pkg_01HX_INET_250M_VOIP_RES",
    "targetPackageVersionId":  "pkv_01HX_INET_250M_V1",
    "targetPackagePrice":      4500.00,
    "currency":                "KES",
    "cycleAnchorPolicy":       "PRESERVE",
    "resolvedEffectiveTiming": "IMMEDIATE",
    "scheduledCommitAt":       null,
    "prorateDelta":            710.00,
    "prorateBillableEventId":  "bil_evt_01HZ_KAMAU_UPGRADE_PRORATE",
    "firstCycleCharge":        null,
    "firstCycleBillableEventId":null,
    "oldCycleCreditAmount":    null,
    "oldCycleCreditBillableEventId":null,
    "cycleStart":              "2026-05-01",
    "cycleEnd":                "2026-05-31",
    "cycleAnchorDayBefore":    1,
    "cycleAnchorDayAfter":     1,
    "cycleDaysRemainingAtUpgrade": 11,
    "billingFrequencyChanged": false,
    "newEquipmentBindings":    [],
    "unboundEquipment":        [],
    "upgradedAt":              "2026-05-20T14:02:18+03:00",
    "initiatingActorUserId":   "cc-agent-wik-007",
    "initiatingActorRole":     "CALL_CENTER"
  }
}
```

7. Cross-DD coordination

7.1 Upstream callers

Caller	upgradeTrigger	Role
Customer portal	`CUSTOMER_REQUESTED_UPGRADE`	`CUSTOMER_SELF_SERVICE` (subject to config)
Call Center	`CUSTOMER_REQUESTED_UPGRADE`	`CALL_CENTER`
BO admin	`ADMIN_UPGRADE`	`SUBSCRIPTION_ADMIN` or `SUBSCRIPTION_OPERATOR`

7.2 Sibling modules

Module	Direction	Contract
SUB-WF-FRAMEWORK-01 (parent)	This DD composes framework primitives	`subscription_operation` row + BPMN start
SUB-WF-FRAMEWORK-01-WORKERS	This DD's BPMNs reference workers by topic	Workers used: `drools-decide`, `sub-lm-put-pending-status`, `sub-lm-compute-cycle-window`, `bil01-emit-intent`, `bil01-charge-pre-authorize?`, `bil01-confirm-charge?`, `ful-call-modify`, `osr-equipment-bind?`, `osr-equipment-unbind?`, `sub-lm-put-master-fields`, `sub-lm-commit-state`, `kafka-emit`, `notification-send`, `subscription-operation-finalize`
SUB-LM-01	Workers write status + master fields	Status flip + package_ref/version + previous_package_ref/version
SIP-01	Workers read Package + Package Version catalog	Target Package validation
PLM-CFG-01	Workers read Service + service_class	Equipment-requirement delta computation
RLM-CFG-01	Workers read HomePass `service_management_endpoints` + `status_code` + status-code catalog	HomePass physical-capability check (R-TP-5/-6/-7); HomePass row is READ-ONLY in this workflow
BIL-01	`bil01-emit-intent` + conditional charge-pre-authorize/confirm	`UPGRADE_INTENT` with proration
BIL-CFG-01	BillableEvent catalog	Upgrade-prorate definitions
FUL-03	`ful-call-modify` worker	`MODIFY_SERVICE` intent. FUL-03 owns the per-service-class fan-out: diff source vs target Package compositions, dispatch `modifyService`/`activate`/`deactivate` to appropriate provisioner per service class, using HomePass `service_management_endpoints` for node + port resolution. Network-side activation (IGMP-snooping, multicast group provisioning, AAA channel registration) handled by FUL-03's provisioner adapters.
OSR-INSTANCE-01	`osr-equipment-bind` / `osr-equipment-unbind` workers	Equipment binding mutations
NOT-01	`notification-send` worker	Customer notification per operator config
BIL-02 / CVM / EM-04	Consume Kafka events	Cycle accrual at new price, churn analytics, reporting

7.3 Build order

This DD deploys after:

1. SUB-WF-FRAMEWORK-01 (parent + WORKERS) v1.0
2. SUB-LM-01 v1.0
3. SIP-01 v1.0
4. PLM-CFG-01 v1.0
5. RLM-CFG-01 v1.0
6. FUL-03 v1.0
7. BIL-01 (UPGRADE_INTENT support)
8. OSR-INSTANCE-01 v1.0
9. At least one FLOW satellite (FLOW-WIK v1.0)

7.4 Stub debt + open coordination items

- **WUG / WTZ / YASSN FLOW satellites** — pending operator deep dives.
- **BIL-CFG-01** `UPGRADE_INTENT` **BillableEvent definitions** — pending BIL-CFG-01 v1.0 promotion. Proration delta formula is BIL-01 responsibility.

- **PLM-CFG-01** `service_class.requires_equipment` **field** — pending PLM-CFG-01 v1.0 promotion. v0.8 doesn't yet declare this; v1.0 must add it for this DD's equipment-delta computation to work.
- **FUL-03** `MODIFY_SERVICE` **intent** — pending FUL-03 v1.1 promotion to add this verb explicitly with per-service-class fan-out (current FUL-03 v1.0 supports restriction-related intents; `MODIFY_SERVICE` is a new verb requested by this DD + DOWNGRADE-01).
- **Equipment REPLACE upgrade case** — out of scope for v1.0. v1.0 handles equipment ADD (new service class requires new equipment, e.g., double→triple-play STB binding). Equipment REPLACE (existing equipment can't deliver the new tier, e.g., ONT generation upgrade for higher speed) is a future scenario. When added in v1.1, the workflow will require source disconnect WO + target install WO at the same HomePass, mirroring RELOCATION-01's two-WO orchestration. Pending workshop confirmation of when KE needs this.

B — Current TZ

Status: To be filled in after codebase cartography review.

Legacy package upgrade flow exists; specific implementation shape — package-versioning behavior, proration delta calculation, equipment-binding propagation, network reconfiguration sequencing — requires source-code review to document accurately. Until that cartography lands, this DD does not assert specific legacy structural claims.